

Problema - Transferencia de Calor 1D en una Aleta

Transferencia de Calor 1D

Se tiene una aleta rectangular unidimensional con conductividad térmica **K** y generación de calor **q** dependiente linealmente de la temperatura.

$$\frac{\partial}{\partial x} \left(k \frac{\partial T}{\partial x} \right) - \frac{hP}{A} (T - T_{\infty}) + q = 0$$



siendo P el perímetro, A el área transversal. Notar que la ecuación se encuentra expresada por unidad de volumen.

Problema adimensionalizado

La expresión anterior puede adimensionalizar definiendo los siguientes términos:

$$\theta = \frac{T - T_{\infty}}{T_b - T_{\infty}} \quad \text{Temperatura adimensional}$$

$$N = \sqrt{\frac{hPL^2}{k_0A}} \quad \text{Parámetro de convección conducción}$$

$$G = \frac{q_0A}{hP(T_b - T_{\infty})} \quad \text{Número de generación de calor}$$

$$\epsilon_G = \epsilon(T_b - T_{\infty}) \quad \text{Parámetro adimensional de generación de calor}$$

$$\epsilon_C = \beta(T_b - T_{\infty}) \quad \text{Parámetro adimensional de conductividad térmica}$$

Problema adimensionalizado

La expresión anterior puede adimensionalizar definiendo los siguientes términos:

$$F = \frac{\partial}{\partial x^*} \left((1 + \epsilon_c \theta) \frac{\partial \theta}{\partial x^*} \right) - N^2 \theta + N^2 G (1 + \epsilon_G \theta) = 0$$

Condiciones de borde

$$\theta(x^* = 0) = 1$$

$$\left. \frac{d\theta}{dx^*} \right|_{x^*=1} = 0$$

$\epsilon_c = 0,4$: Parámetro adimensional de conductividad térmica

$\epsilon_G = 0,6$: Parámetro adimensional de generación de calor

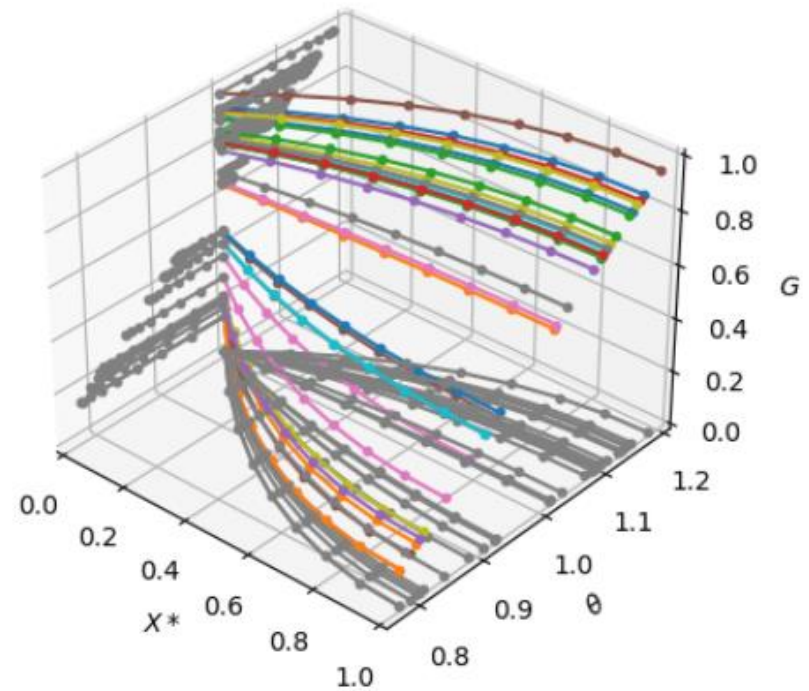
$N = 1$: Parámetro de convección-conducción

Datos del problema físico

$$F = \frac{\partial}{\partial x^*} \left((1 + \epsilon_c \theta) \frac{\partial \theta}{\partial x^*} \right) - N^2 \theta + N^2 G (1 + \epsilon_G \theta) = 0$$

La información proporcionada consiste en un dataframe de 500 muestras de nueve temperaturas equiespaciadas en $x^* \in [0,1]$, junto con su correspondiente valor de generación de calor G . Los datos se encuentra proporcionados en el archivo `datos_calor.csv`.

Oommen, Vivek & Srinivasan, Balaji. (2022). Solving Inverse Heat Transfer Problems Without Surrogate Models: A Fast, Data-Sparse, Physics Informed Neural Network Approach. *Journal of Computing and Information Science in Engineering*. 22. 1-12. 10.1115/1.4053800.



Modelo subrogado

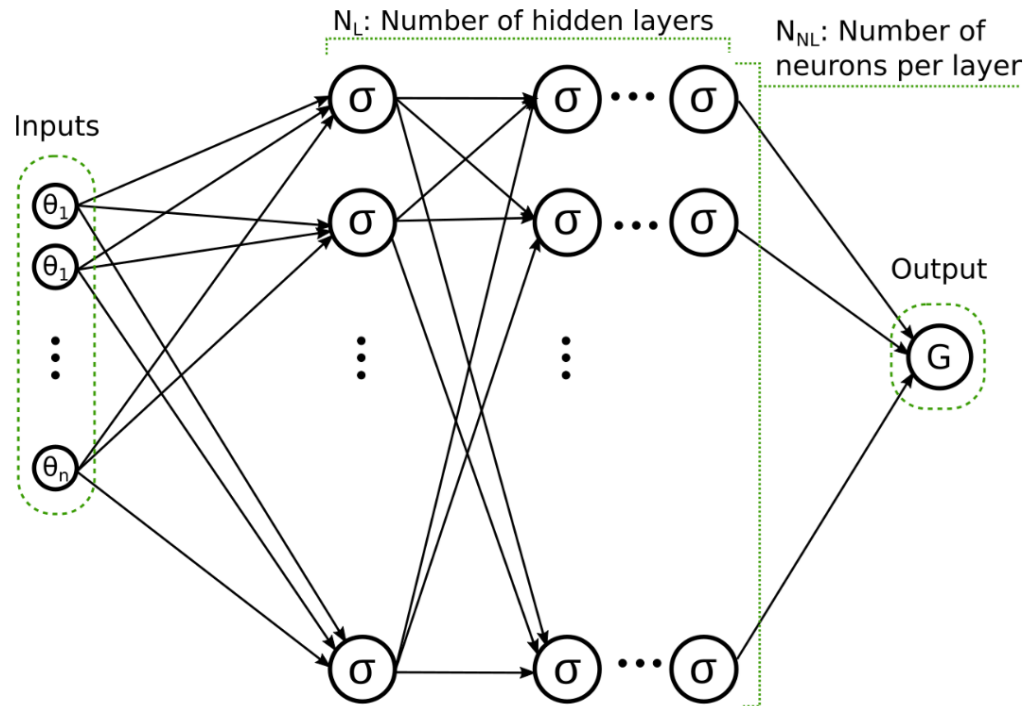
Un modelo Subrogado es una red neuronal que actúa como modelo sustituto de un sistema físico complejo, aproximando soluciones de ecuaciones diferenciales con alta fidelidad y bajo costo computacional a partir de datos de muestra..

- Reemplaza simulaciones costosas (FEM, CFD, etc.)
- Respeta las leyes físicas del sistema proveniente de los datos provistos.
- Acelera el análisis en procesos de:
 - optimización
 - control en tiempo real
 - Diseño

asistido

Modelo subrogado

Un modelo Subrogado es una red neuronal que actúa como modelo sustituto de un sistema físico complejo, aproximando soluciones de ecuaciones diferenciales con alta fidelidad y bajo costo computacional a partir de datos de muestra..



Normalización de datos

Es una práctica útil para mejorar el proceso de aprendizaje. Existen diferentes técnicas para normalizar los datos disponibles.

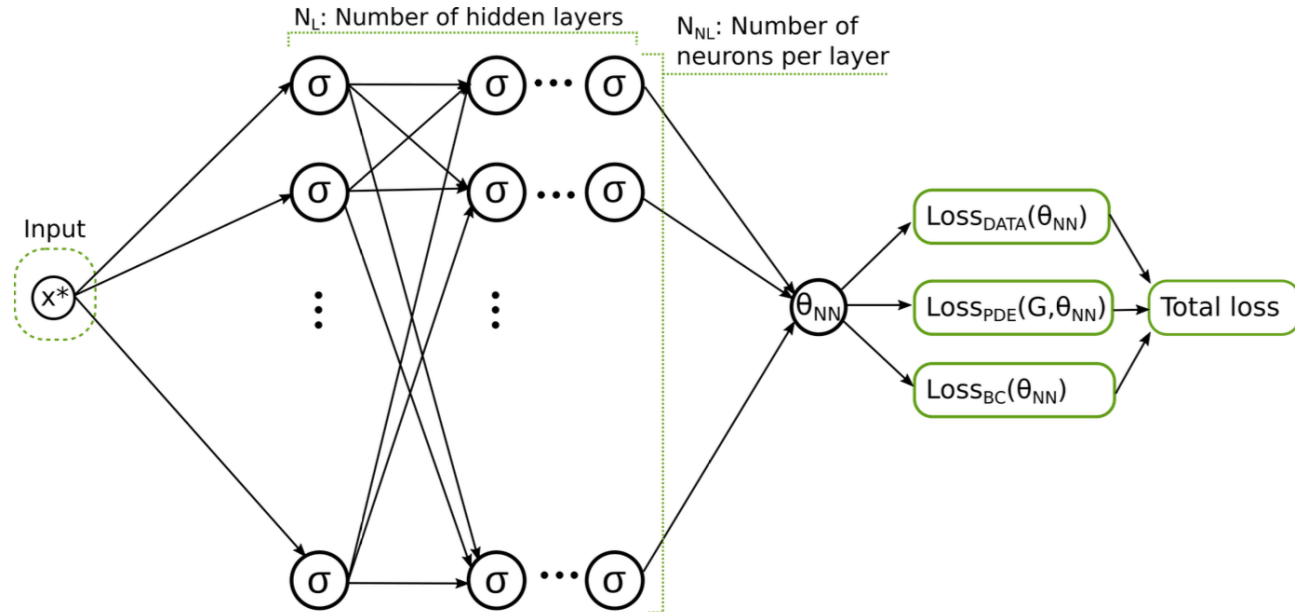
Escalado MIN-MAX: Permite fijar los datos dentro de la escala del valor máximo y mínimo de la serie de datos disponibles ([0,1] o [-1,1]).

Z-score (desviación estandar): Ordena la serie de datos con media 0 y desviación estándar 1.

Normalización MIN-MAX [-1, 1]

$$x_{norm} = 2 \frac{x - x_{min}}{x_{max} - x_{min}} - 1$$

PINN - Transferencia de Calor



$$F = \frac{\partial}{\partial x^*} \left((1 + \epsilon_c \theta) \frac{\partial \theta}{\partial x^*} \right) - N^2 \theta + N^2 G (1 + \epsilon_G \theta) = 0$$

$$MSE_F = \frac{1}{N_f} \sum_{i=1}^{N_f} |F(\theta_{NN}^i)|^2 \quad MSE_\theta = \frac{1}{N_\theta} \sum_{i=1}^{N_\theta} F |(\theta_{data}^i - \theta_{NN}^i)|^2 \quad MSE_{\frac{\partial \theta}{\partial x}} = \left| \frac{\partial \theta}{\partial x} \right|_{x^*=1}^2$$

Optimizador ADAM

Es un optimizador de primer orden, ya que se enfoca en seguir la tendencia del gradiente para encontrar un máximo o un mínimo. Combina las ventajas de dos métodos::

- AdaGrad, que adapta la tasa de aprendizaje por parámetro.
- RMSProp, que utiliza una media móvil del cuadrado de los gradientes.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \rightarrow \textit{Media del gradiente}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \rightarrow \textit{Varianza del gradiente}$$

$$\theta_{t+1} = \theta_t - l_r \frac{m_t}{\sqrt{v_t} + \epsilon}$$

$g_t \rightarrow \textit{Gradiente}$

$l_r \rightarrow \textit{Tasa de aprendizaje}$

$\beta_1, \beta_2 \rightarrow \textit{Factores de decaimiento}$

$\epsilon \rightarrow \textit{Término de estabilidad numérica}$

Optimizador ADAM

```
CLASS torch.optim.Adam(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0,
    amsgrad=False, *, foreach=None, maximize=False, capturable=False, differentiable=False,
    fused=None, decoupled_weight_decay=False) \[SOURCE\]
```

input : γ (*lr*), β_1, β_2 (*betas*), θ_0 (*params*), $f(\theta)$ (*objective*)
 λ (*weight decay*), *amsgrad*, *maximize*, ϵ (*epsilon*)

initialize : $m_0 \leftarrow 0$ (first moment), $v_0 \leftarrow 0$ (second moment), $v_0^{max} \leftarrow 0$

for $t = 1$ **to** ... **do**

if *maximize* :

$g_t \leftarrow -\nabla_{\theta} f_t(\theta_{t-1})$

else

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$

if $\lambda \neq 0$

$g_t \leftarrow g_t + \lambda \theta_{t-1}$

$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$

$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$

$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$

if *amsgrad*

$v_t^{max} \leftarrow \max(v_{t-1}^{max}, v_t)$

$\widehat{v}_t \leftarrow v_t^{max} / (1 - \beta_2^t)$

else

$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$

$\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$

return θ_t

Optimizador LBFGS

```
CLASS torch.optim.LBFGS(params, lr=1, max_iter=20, max_eval=None, tolerance_grad=1e-07,  
    tolerance_change=1e-09, history_size=100, line_search_fn=None) \[SOURCE\]
```

L-BFGS (Limited-memory Broyden-Fletcher-Goldfarb-Shanno) es un algoritmo de optimización de segundo orden que pertenece a la familia de los métodos Quasi-Newton. Estos métodos están diseñados para encontrar el mínimo de una función objetivo utilizando información derivada de su gradiente (curvatura), pero sin necesidad de calcular explícitamente la matriz Hessiana (que contiene segundas derivadas).

- Los métodos de segundo orden, como el de Newton, usan la Hessiana para determinar la dirección y el tamaño del paso óptimo hacia el mínimo, lo que es muy costoso computacionalmente.
- L-BFGS evita este costo aproximando la inversa de la Hessiana utilizando únicamente las últimas iteraciones del gradiente, lo que reduce notablemente la necesidad de almacenamiento y cálculo.

Optimizador LBFGS

```
CLASS torch.optim.LBFGS(params, lr=1, max_iter=20, max_eval=None, tolerance_grad=1e-07,  
    tolerance_change=1e-09, history_size=100, line_search_fn=None) \[SOURCE\]
```

Broyden, Fletcher, Goldfarb, Shanno



Implementaciones

1. Modelo subrogado → [Ejemplo 3.1](#)
2. PINN: Optimizador ADAM y LBFGS → [Ejemplo 3.2](#)